NANYANG
THE INNOVATIVE POLYTECHNIC

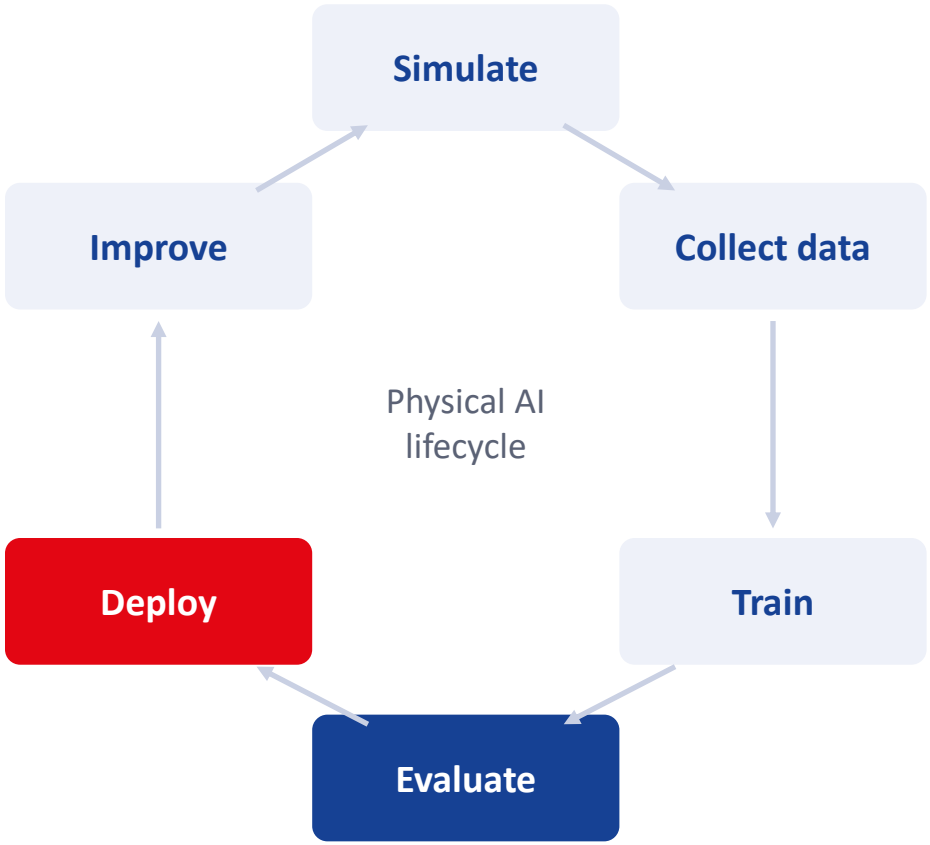
Reinforcement Learning II

From simulation to the real arm

Data Acquisition & Policy Learning · Lesson 7



Where we are: Block D · taking an RL policy to the real arm



A	Foundations L1–L2
B	Collecting robot data L3–L4
C	RL in simulation L5–L6
D	**RL: sim → real arm L7–L8**
E	**Imitation learning L9–L10**
F	**Evaluate & deploy IL L11–L12**
G	**Vision-language-action L13–L14**

Entry check: three questions on Lesson 6

1. In Lab 6, what happened when both smoothness penalties were removed? What did the reward curve show?
2. What units does Isaac Lab use for joint positions? What units does LeRobot report on the real arm?
3. Your unit-mapping table records four things for every joint. What are they?

5 minutes, on paper or in the poll. We go through the answers together.

Keep open today: your Lab 6 unit-mapping table and your reward-variant videos. We use both in Part 2.

Learning outcomes

By the end of this lesson you should be able to:

1. **Explain the gap**
   why a policy that works in simulation can fail on the real SO-101

2. **State the contract**
   the deployment contract: observation order, units, scaling, rate and action limits

3. **Judge transfer**
   why a Reach policy can transfer while a Lift policy needs extra perception

4. **Evaluate on the arm**
   which metrics and failure categories to use for an RL policy on the real arm

Prediction: what will your policy do on the real arm?

THE SITUATION

Your randomised Reach policy from Lab 6 lands within 1 cm of every target in simulation.

Next week you run it, unchanged, on your pod's real SO-101.

What happens?

A Same as simulation: within 1 cm of every target

B It reaches, but less accurately: 2 to 4 cm off

C It moves wildly or slams a joint into its limit

D It barely moves at all

Vote now and write your letter down. We come back to it at the end of the lesson.

Part 1

The reality gap on a low-cost arm

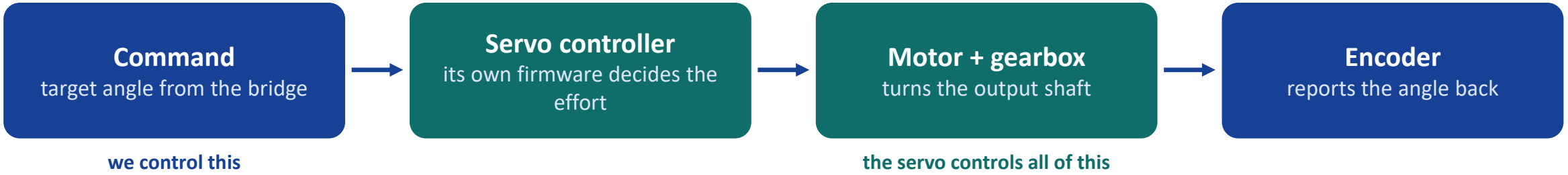
Why a policy that works in simulation can fail on the real SO-101

Same policy, two different arms

	In Isaac Lab	On the real SO-101
Actuators	Ideal drives that follow the stiffness and damping you set, every time	Hobby servos with backlash and friction; response changes with load; position commands only
Sensing	Exact joint angles and velocities, with no noise unless you add it	Encoder steps, a small calibration offset and noisy readings; velocity must be estimated
Timing	Observe and act in the same instant, exactly 30 times a second	Reading and writing six servos over USB takes time, and the loop rate can slip
Target	Known exactly, in the robot's own frame	Must be chosen and measured in the robot's frame; the grid mat is our ground truth

The policy only ever met the left column. Everything on the right is new to it.

Inside a hobby servo: where the dynamics gap comes from



Backlash

The gears have slack. Reverse direction and the motor turns a little before the shaft moves.

Friction

Very small commands may not move the joint at all. Then it suddenly unsticks.

Load

The same command moves faster folded up than stretched out, where gravity pulls harder.

Position only

You send an angle, never a force. The servo decides how hard to push to get there.

Simulation models each servo as a clean spring and damper. Close, but not the same machine: that difference is the dynamics gap.

Worked example: how big is a “small” error at the gripper?

Error at the gripper ≈ distance from the joint × angle error (in radians)

A 1° calibration offset at the shoulder

1° = 0.0175 rad
× 0.30 m from shoulder to gripper

≈ 5 mm

at the gripper, from one degree

One step of delay, joint moving at 1 rad/s

0.033 s × 1 rad/s = 0.033 rad (about 1.9°)
× 0.30 m from shoulder to gripper

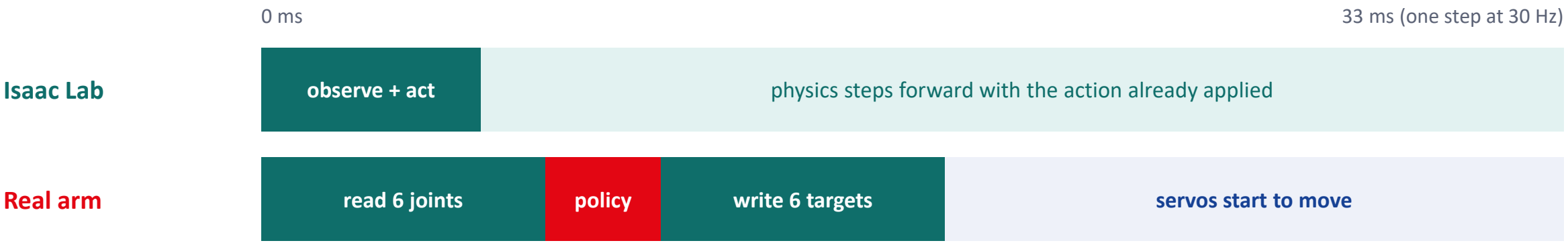
≈ 10 mm

of stale information, every step

Our success tolerance is 20 mm. Two small errors like these use up most of it before the policy makes a mistake of its own.

0.30 m is a round figure for the SO-101's reach, used for illustration.

Sensing, delay and control rate



Segment lengths are illustrative. You measured your arm's real loop rate at the Lab 6 arm station.

Noise

Readings jitter by an encoder step or two, even when the arm is still.

Offset

The calibration zero is slightly off: a constant bias on every reading.

Delay

The policy acts on where the arm was, not where it is now.

Rate

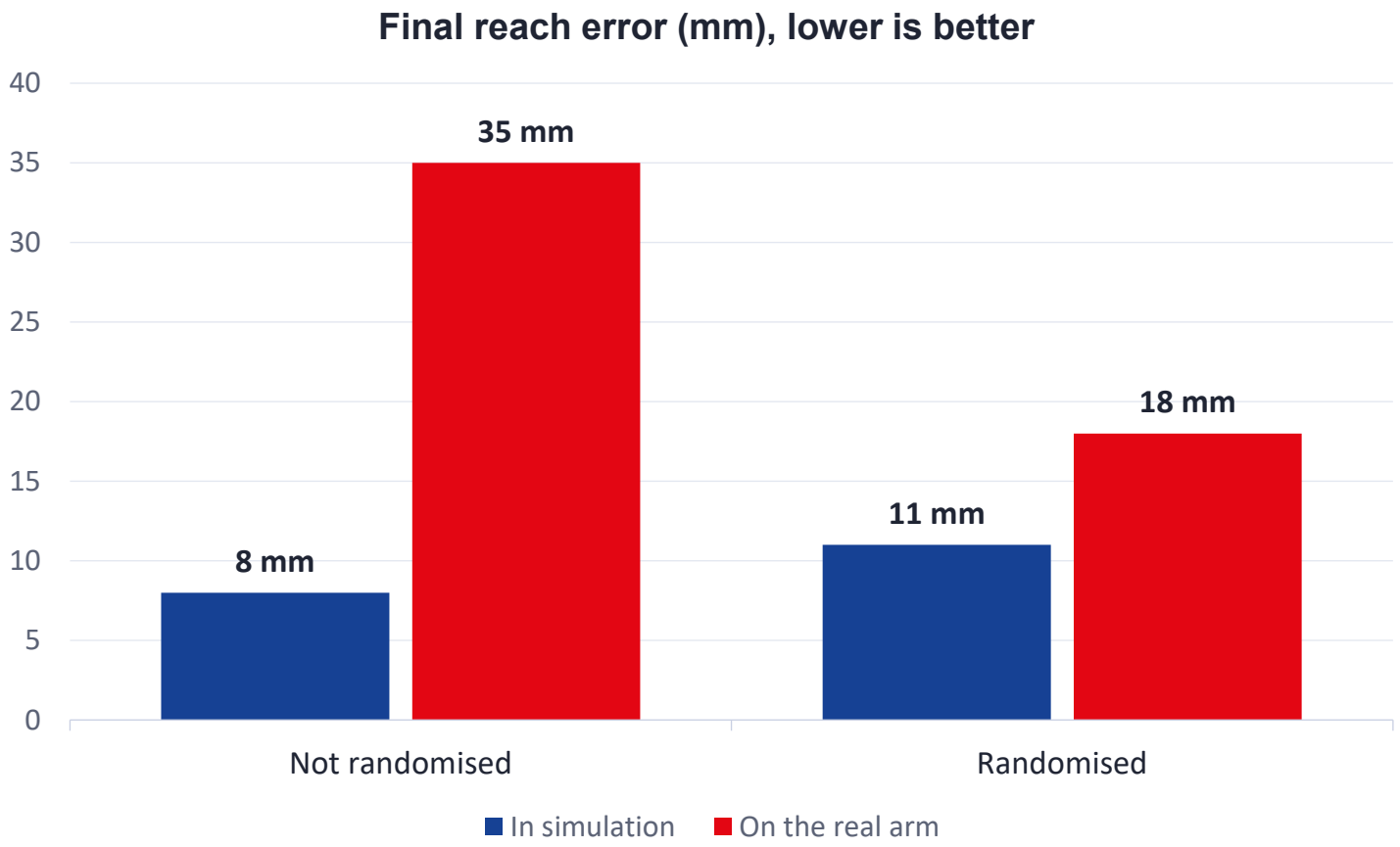
If the loop slips from 30 Hz to 20 Hz, each action lasts 50 ms instead of 33.

What helps: train on a world that is a little bit wrong

Gap	What we randomise or change in training	What the policy learns
Servo strength dynamics	Actuator stiffness and damping drawn from a range each episode	**Don't rely on one exact response**
Mass and friction dynamics	Link mass and joint friction varied at every reset	**Push through sticky or heavy joints**
Noise and offsets sensing	Noise on joint readings (already on in Reach), plus a small random bias	**Trust the trend, not the last decimal**
Delay timing	Hold each action or observation back by 0–2 steps at random	**Don't expect an instant response**
Aggressive moves all	A conservative action scale; keep the action-rate penalty	**Take steps the servo can follow**

All of these live in the task config: the same events and noise settings you switched on in Labs 4 and 6.

The price and the payoff of randomisation



Illustrative numbers. You measure your pod's own in Lab 8.

Slightly worse in simulation

A generalist gives up a little precision on the one perfect world.

Much better on the real arm

The real arm looks like one more variation it has already practised on.

Too wide hurts

Huge ranges and the policy struggles to learn at all: the Lesson 3 trade-off.

Fail cheap: test in a changed simulator first

1

Train

Isaac Lab, randomisation on

Cost of failure: nothing

→

2

Changed simulator

Heavier joint noise, different actuator gains, extra delay: a world it never trained on

Cost of failure: nothing

→

3

Real arm, slowly

Reduced step size, instructor present, a hand near the power switch

Cost of failure: a stopped run

→

4

Real arm, normal limits

Five targets per policy, every result logged

Cost of failure: a logged failure to learn from

Fails in the changed simulator?

It will almost certainly fail on the arm too. Fix it here, where it costs nothing.

Passes?

It can still fail on the arm, but for fewer reasons, and you know which ones are left.

Part 2

Getting the policy out

Export, the deployment contract and the bridge loop

Export: what leaves the simulator

Training checkpoint

`model_1000.pt`

actor (the policy)

critic (the coach)

optimiser state

observation normaliser, if used

export

`policy.pt` TorchScript

Actor + normaliser in one file. Load with torch.jit.load and run it in Python with PyTorch.

`policy.onnx` ONNX

The same network in a framework-neutral format. Runs with ONNX Runtime, even without PyTorch.

The critic stays behind: it only coached training. On the arm, only the actor runs.

Isaac Lab's play step writes both files to the run's exported/ folder. The normaliser is already inside: don't apply it twice.

The deployment contract

1 · Observation order

joint positions (6) → joint velocities (6) → target pose (7) → last action (6) = 25 numbers

2 · Units

radians, rad/s and metres; target orientation as a quaternion in (w, x, y, z) order

3 · Scaling and offsets

joint positions relative to the default pose; normaliser already inside the exported file

4 · Control rate

30 Hz: one decision every 33 ms, the rate it was trained at

5 · Action scaling

joint target = default pose + scale × action, with the scale from your config

6 · Joint order

shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper

Every item must match training exactly. One mismatch is enough to make a good policy look broken.

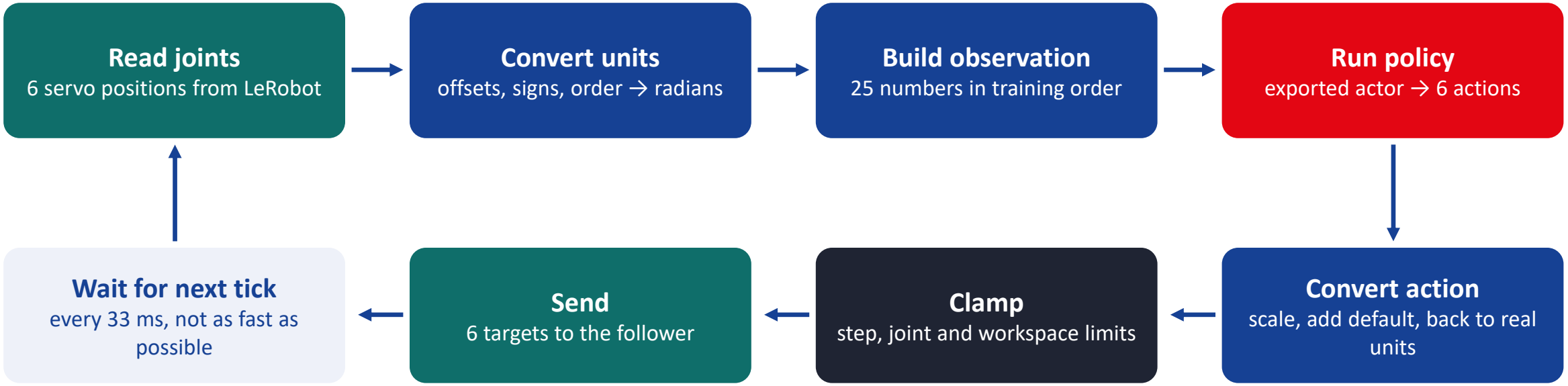
Worked example: one joint through the contract

Unit-table row: elbow_flex · real units: degrees · zero offset: 10° · sign: −1 · default pose: 0.0 rad

1	**Read**	LeRobot reports the real joint angle	**45.0°**
2	**To radians**	−1 × (45.0 − 10) × π/180	**−0.611 rad**
3	**Observation**	position − default pose = −0.611 − 0.0	**−0.611**
4	**Policy**	the network's output for this joint	**a = −1.0**
5	**Action → target**	default + 0.5 × a = 0.0 + 0.5 × (−1.0)	**−0.500 rad**
6	**Back to degrees**	10 + (−1) × (−0.500) × 180/π	**38.6°**
7	**Clamp**	change = 38.6 − 45.0 = −6.4°, limit ±5° per step	**send 40.0°**

Illustrative values, with an illustrative action scale of 0.5. Use your pod's unit table and the scale in your config.

The bridge loop: one decision every 33 ms



Teal talks to hardware in real units. **Red** is the policy, which only speaks radians. **Blue** translates between them.

Almost every contract bug lives in a blue box. Your Lab 6 unit table is the code behind the two “convert” boxes.

Symptom → suspect: contract bugs look like bad policies

WHAT YOU SEE

Moves away from the target, then pins one joint at its limit

Suspect: the sign of that joint is flipped in the unit table

WHAT YOU SEE

Heads to a strange pose first, then roughly reaches

Suspect: absolute angles sent instead of positions relative to the default pose

WHAT YOU SEE

Reaches, but jerky, with overshoot

Suspect: the loop runs faster or slower than the 30 Hz it trained at

WHAT YOU SEE

Barely moves at all

Suspect: action scale applied twice, or a clamp so tight that every step is cut off

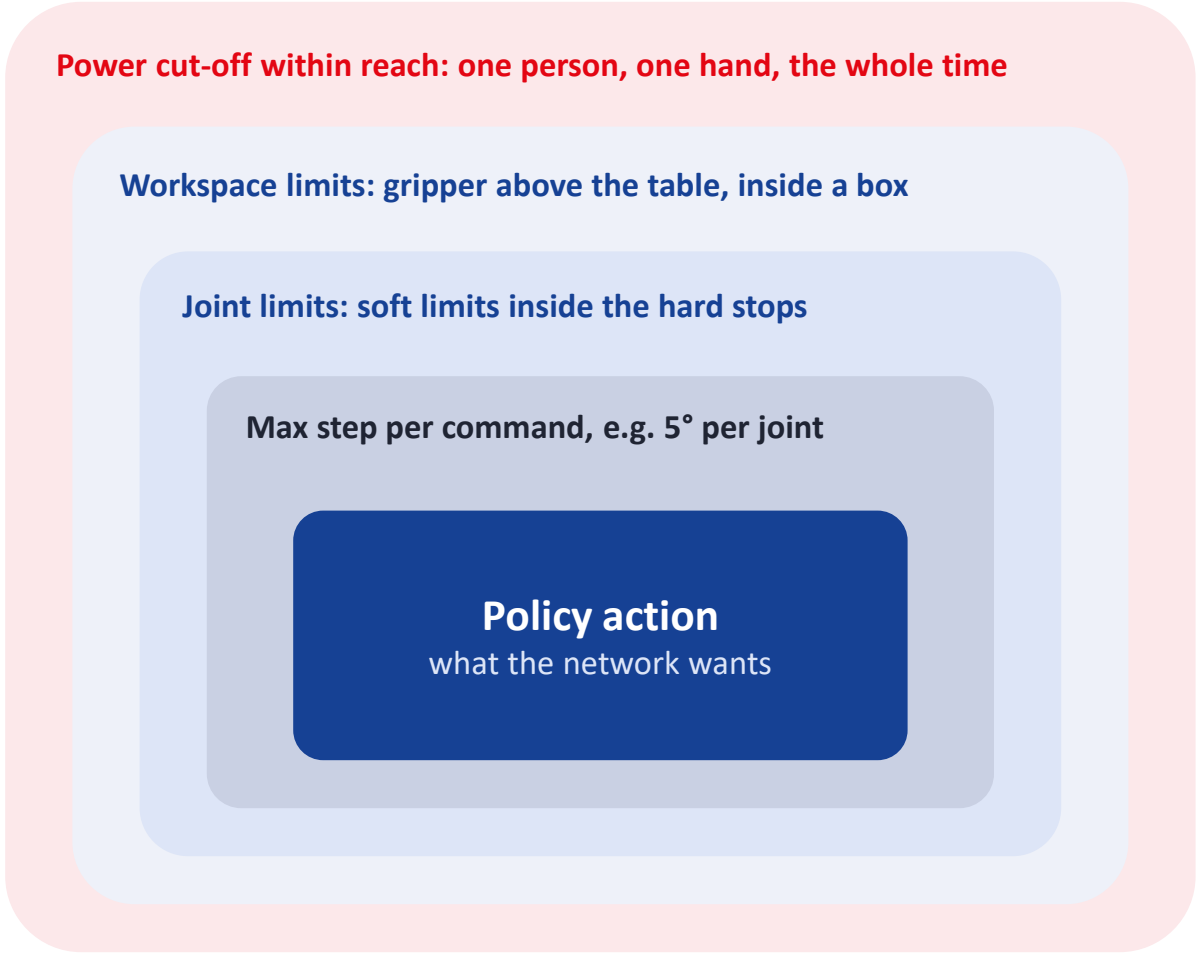
Check the contract before you blame the policy.

Why Reach transfers and Lift does not (yet)

Observation term	Reach	Lift	On the real arm, it comes from…
Joint positions	**Yes**	**Yes**	the servo encoders
Joint velocities	**Yes**	**Yes**	change in position ÷ time
Last action	**Yes**	**Yes**	the bridge remembers it
Target to reach	**Yes**	**Yes**	we choose it, in the robot's frame
Cube position and orientation	**No**	**Yes**	**a camera + pose estimator, with its own error and delay**

Rule: a policy can only transfer if everything it observes can be measured on the real robot.

The safety envelope



First runs

Half the usual step limit

Instructor present for the first command

Hands out of the follower's path, always

One person whose only job is the power switch

In LeRobot

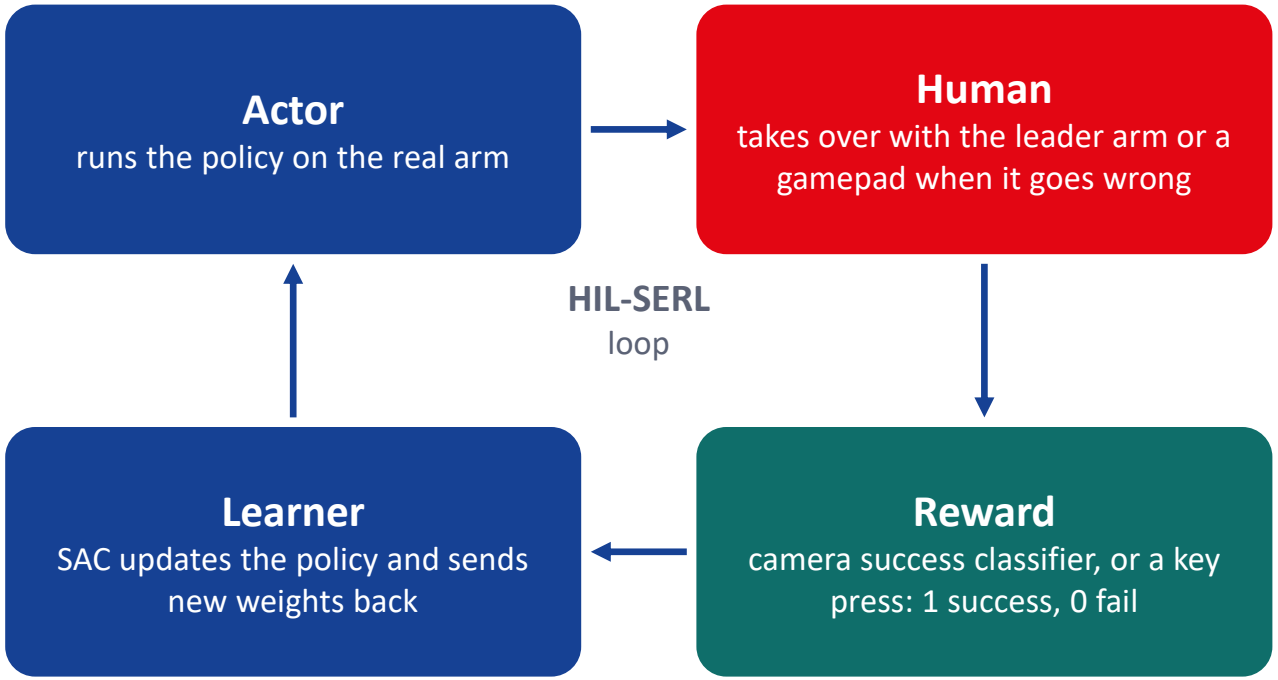
`max_relative_target` caps how far each motor may move in one action. Same job as our clamp.

Part 3

RL on real hardware and how to evaluate RL

Human-in-the-loop RL, metrics and failure categories

Human-in-the-loop RL on the real arm (HIL-SERL)



Luo et al. (2024): near-perfect success on real manipulation tasks after 1–2.5 hours of training.

Explained, not practised

An exploring policy on a real arm needs one person watching one arm, the whole time.

With pods of 3–4 learners and a full class, we cannot supervise that safely.

Workspace bounds have to be recorded first, so exploration stays inside a box.

So in this unit, RL learns in simulation and only the finished policy touches the arm.

Metrics for Reach on the real arm

12 mm

Final position error

distance from gripper to target when the episode ends

4 / 5

Success

final error inside the 20 mm tolerance, reported as a rate

1.4 s

Time to target

first moment the gripper is inside the tolerance

3

Smoothness

e.g. direction reversals per joint; fewer is smoother

Example values only.

From the joints

The bridge runs forward kinematics on the joint readings. Exact and automatic, but it trusts the calibration.

From the grid mat

Check the gripper against the marked target with a ruler. Slower, but it catches calibration errors.

Worked example: scoring five targets

Target	Final error	Within 20 mm?	Time to target
1	8 mm	Yes	1.2 s
2	14 mm	Yes	1.6 s
3	23 mm	No	—
4	11 mm	Yes	1.4 s
5	31 mm	No	—

3 / 5 = 60%
success rate

17.4 mm
mean final error

14 mm
median final error

Five trials is a tiny sample. A 95% confidence interval for 3 / 5 runs from about 23% to 88%. Lesson 11 shows you how to compute it.

Five ways a Reach policy fails on the arm

Overshoot

You see: goes past the target, then comes back

Likely cause: action scale too large, or less damping on the real servos

Oscillation

You see: wobbles around the target and never settles

Likely cause: more delay than in training, or no smoothness penalties

Joint limit hit

You see: a joint drives to its end stop and stays

Likely cause: sign flip, unreachable target, or sim limits wider than real

Drift

You see: results creep further off as the session goes on

Likely cause: servos warming, sagging under load, or calibration slipping

Transfer failure

You see: looks nothing like the simulation video

Likely cause: a contract mismatch: units, order, scaling or rate

Log every failure

target · category · video timestamp · likely cause · what you will check next

Diagnose the clip

Clip A

VIDEO (optional)
record on the real arm beforehand

The arm gets within 1 cm, then wobbles back and forth around the target until the episode ends.

Category? Likely cause?

Clip B

VIDEO (optional)
record on the real arm beforehand

Every target is reached, but over 40 minutes the gripper finishes a little lower on each run.

Category? Likely cause?

Clip C

VIDEO (optional)
record on the real arm beforehand

On the very first command, the base swings the wrong way and stops hard against its limit.

Category? Likely cause?

Pairs, 4 minutes. Vote on the category, then argue for the cause.

Activity: contract check

Trained with (env config)

Rate 30 Hz

Joint order shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper

Observation joint positions relative to default, rad (6) · joint velocities, rad/s (6) · target pose, m + quaternion (7) · last action (6)

Total 25 numbers

Action target = default + 0.5 × action, in rad

What the bridge does: find 3 bugs

Rate 30 Hz

Joint order shoulder_pan, shoulder_lift, elbow_flex, wrist_roll, wrist_flex, gripper

Observation joint positions relative to default, deg (6) · joint velocities, rad/s (6) · target pose, m + quaternion (7)

Total 19 numbers

Action target = default + 0.5 × action, in rad → degrees → clamp ±5° per step

Pairs, 8 minutes: circle each bug, then predict what the arm would do because of it.

Back to your prediction

A	**Same as simulation**	Unlikely: it would mean there is no reality gap at all.
B	**Reaches, less accurately**	The expected result with a correct bridge. That error is the reality gap; randomisation shrinks it.
C	**Moves wildly or hits a limit**	A contract mismatch: units, sign or joint order. The safety envelope stops it doing harm.
D	**Barely moves**	Scaling or clamping: the action is shrunk or cut off on its way to the servo.

B is a physics problem: you shrink it with training. C and D are bookkeeping problems: fix them at your desk before you power the arm.

Exit ticket

1. List four items of a deployment contract.
2. Explain why a Reach policy can transfer to the real arm while a Lift policy cannot.
3. Name two failure modes, and give a likely cause for each.

On your own, on paper, about 5 minutes.

Before Lab 8: finish your unit-mapping table. The bridge cannot run without it.

Next week: L8 lab · run the Reach policy on the SO-101

Sim station · export and sim-to-sim check

- Export both Lab 6 policies (randomised and not)
- Play each in a changed simulator: heavier joint noise, different actuator gains
- Record the mean final error over 20 targets
- Predict which will do better on the real arm, and why

Arm station · Reach on the real arm

- Load each policy into the bridge with your unit table
- First run at reduced step size, instructor present
- Command 5 targets per policy and record the error
- Log every failure with its category from today

Bring: your unit-mapping table, both checkpoints and your safety limits. Before anyone powers an arm, your pod states its unit table and limits to the instructor.

References and further reading

Tobin, J. et al. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. https://arxiv.org/abs/1703.06907

Peng, X. B. et al. (2018). Sim-to-real transfer of robotic control with dynamics randomization. https://arxiv.org/abs/1710.06537

Tan, J. et al. (2018). Sim-to-real: learning agile locomotion for quadruped robots. https://arxiv.org/abs/1804.10332

Luo, J., Xu, C., Wu, J. & Levine, S. (2024). Precise and dexterous robotic manipulation via human-in-the-loop RL. https://arxiv.org/abs/2410.21845

LeRobot documentation: HIL-SERL guide. https://huggingface.co/docs/lerobot/hilserl

LeRobot documentation: SO-101. https://huggingface.co/docs/lerobot/so101

Isaac Lab: RSL-RL policy exporter (TorchScript and ONNX). https://isaac-sim.github.io/IsaacLab/main/_modules/isaaclab_rl/rsl_rl/exporter.html

isaac_so_arm101 (SO-ARM100/101 Isaac Lab tasks). https://github.com/MuammerBay/isaac_so_arm101

NVIDIA: Train an SO-101 robot from sim-to-real with NVIDIA Isaac. https://docs.nvidia.com/learning/physical-ai/sim-to-real-so-101/latest/index.html